

01

Why your build broke at 80%

What happens in the context window when the AI starts undoing your features. The structural reason, not the vibes.

The moment it turns

You asked for something small. Move a button. Fix a date format. The diff looked clean, the app compiled, and you shipped it.

Then you clicked around and found that uploads were broken. Uploads. The feature you finished on Tuesday. The one that already worked.

So you told the AI to fix uploads. It fixed uploads. Now the date format is wrong again.

If you have been here, you know the feeling that comes next: the creeping suspicion that the tool has turned on you. It has not. It is doing exactly what it was built to do, with exactly the information it has. The problem is structural. It is knowable. And once you can see it, you can beat it.

That is this chapter: the mechanism, in plain terms, and the five habits that stop the bleeding tonight. The rest of the manual builds the actual cure.

What the AI actually remembers

Every AI coding tool, no matter the brand, works inside a **context window**.

CONTEXT WINDOW

The fixed span of text the model can consider at one time: your instructions, the files it read, the diffs it wrote, the errors it saw. Everything outside the window does not exist for it. Not “is harder to recall.” Does not exist.

The window is not memory. It is closer to a transcript, and the transcript has a hard length limit. In 2026, the big models run somewhere between two hundred thousand and a million tokens, where a token is roughly three-quarters of a word. That sounds enormous. A working session burns it fast: every file the tool opens lands in the window, every diff, every stack trace, every “here’s what I found.”

When the transcript fills up, one of two things happens. Old content falls off the end, or the tool compresses it into a summary. The summary keeps what looked important at the time and quietly drops what looked incidental. “Looked” is the word doing all the damage.

And between sessions? Nothing carries over. Close the chat, and that transcript is gone for good. Tomorrow’s session starts with three things: your codebase, your prompt, and total amnesia about how the codebase got that way.

Your features are more than their code

Here is the part almost nobody tells you, and it is the crux of the whole chapter.

A working feature is not just code. It is code plus a set of constraints that make the code correct. “Uploads must stream to storage instead of buffering, because a 500MB video crushes the serverless function.” “Dates must be formatted server-side, because the client’s timezone lies.” The code shows **what** it does. The reason it must be done that strange way lived in one place only: the conversation where you and the AI figured it out.

That conversation is gone.

So three weeks later, a fresh session opens the upload file while working on something unrelated. It sees an oddly complicated streaming implementation where a simple buffer would do. It has no idea about the 500MB video. Cleaning it up is the **correct move** given what it can see. It is not sabotaging your feature. It is tidying code whose reason for existing is written down nowhere.

Code says what. Only the transcript said why. And the transcript is gone.

Multiply that by every feature you have shipped, and you have the real shape of the problem: your app is held together by dozens of invisible rules, and your tool can only see the code.

Why it hits at 80% and not sooner

The wall is not one failure. It is three curves crossing.

1. Your codebase outgrows the window

In week one, the whole project fits in context. The AI is briefly all-knowing: it holds every file at once, and it feels like magic. That magic is what got you hooked, and it is real. It is also temporary. A mid-sized app is a few hundred files and several hundred thousand tokens, and past that point the AI is no longer reading your codebase. It is sampling it.

2. Your constraints pile up

Every feature that works adds rules-with-reasons to the pile: the auth rule, the streaming rule, the timezone rule, the webhook-ordering rule. At 20% built you might have three of these. At 80% you have sixty, they interact, and not one of them is written down. The number of ways to break something grows with every single thing that works.

3. Your sessions multiply

By 80% you are dozens of sessions deep. Each one started amnesiac. Each rebuilt its own partial picture of your app from whatever files it happened to read. Session forty's mental model disagrees with session twelve's, and both of them wrote code you are still running.

The 80% wall is the point where the invisible constraints outnumber what fits in the window. It is not a skill problem. It is arithmetic. Which is good news, because arithmetic can be engineered around.

The regression loop

Once you cross the wall without knowing it, the same spiral catches nearly everyone:

1. You ask for change B. The AI edits, and quietly violates invariant A, which was nowhere in its view.
2. You spot A broken and ask for a fix. The fix trips constraint C, or re-breaks B.

3. Fixes keep compiling, so you start accepting diffs without reading them. Now drift compounds silently.
4. Frustration peaks, and the thought arrives: “maybe I should just have it rebuild the whole thing clean.”

The rewrite is the wall wearing a costume. A rebuild trades bugs you know for bugs you do not, and it resets your pile of hard-won constraints to zero. The pile was the only thing you had earned. Rebuilds stall at their own 60% for exactly the reasons the original stalled at 80.

Two entries from my own build log

I hit this wall with years of engineering behind me, on my own consulting site, the month I am writing this. The wall does not care about your resume.

FROM THE BUILD LOG

The same bug, twice

My site’s framework had a quirk: the renderer ate the space after bold text in certain conditions, so “\$20M+ in client revenue” shipped to production as “\$20M+in client revenue.” A session found it, fixed it, and I wrote the lesson in the project’s docs.

Weeks later, a different session reintroduced the exact same bug in new copy. It had never read the lesson. Writing it down was not enough, because prose only works on readers, and the next session is not guaranteed to be one.

What finally stuck was a one-line check command in the repo that fails loudly whenever the pattern appears, run before every release. The bug never shipped again. Rules a machine can run beat rules a reader must remember. That idea shows up in every chapter of this manual.

FROM THE BUILD LOG

The demo that lied for weeks

That same site had three lead forms: contact, a sample-chapter signup, and a beta waitlist. All three worked flawlessly in the demo. In production, for weeks, every single submission fell into a server log nobody reads, because one environment variable, the email key, had never been installed on the live host.

No error. No bounce. The page told every visitor “Got it.”

I only found out because I tested a **new** feature end to end on the live site, and that test failed loudly enough to make me look. Two lessons, and they get their own chapter later: production is a different machine than the demo, and “it works” is a claim about the path you actually tested, never about the code you wrote.

What does not fix it

You will be tempted by four false exits. I have tried all four.

A smarter model. Smarter per-glance, same amnesia. Drift is not caused by low intelligence. It is caused by what is out of view.

A bigger window. A million tokens digests a bigger codebase. It does not conjure the sixty reasons-why that were never written anywhere a window could hold.

Telling it to remember. “Please always keep uploads streaming” pins the rule to one transcript. Politeness is not persistence. The next session never heard you.

Pasting everything, every time. Stuffing the repo into every prompt burns the window on **what** while still containing zero **why**, and it crowds out the space the model needs to think.

What works: memory the window cannot lose

The move that changes everything is small: stop treating the conversation as storage.

Anything that must survive the session goes in a file, in the repo, because files are the only thing every future session is guaranteed to be able to see. Your job quietly changes at the wall. You stop being the person who asks for features and start being the keeper of the memory the tool does not have.

Five habits, starting tonight:

- ☐ **Write the invariant list.** One file in the repo root. One line per rule-with-a-reason: “Uploads stream, never buffer: 500MB videos kill the function.” Walk your app for thirty minutes and write down every “must” you can remember. Most AI tools automatically read a file named CLAUDE.md or AGENTS.md, and that is exactly where this belongs.

- ☐ **Open every session by pointing at that file.** Before the task, not after the damage: “Read the invariants file first.” Ten seconds that replaces the memory of every session before this one.
- ☐ **Read every diff before you accept it.** The moment you stop reading is the moment drift starts compounding. One rule of thumb catches most of it: if the diff touches a file your change had no business touching, stop and ask why.
- ☐ **One change per commit, committed the moment it works.** Small commits are restore points. Rolling back beats an apology prompt every single time, and it protects the constraint pile you have earned.
- ☐ **Turn every caught bug into a written rule, same day.** When the AI breaks an invariant, fixing the code is half the job. The other half is adding the rule to the file, and, wherever possible, turning it into a check a machine runs. That is the build-log story above. No other habit in this manual pays back more per minute spent.

These five stop the bleeding. They are triage, not the cure. The cure is a single page the AI re-reads at the start of every session, one that carries your architecture, your constraints, and your definition of done. Building that page is the next chapter, and it is the reason this manual exists.

The wall is not proof you can't do this. It is the point where the tool's memory ran out, and yours has to take over: on paper, in the repo, where every future session finds it.

NEXT • CHAPTER 02

The spec is the moat

The one-page spec the AI keeps re-reading. Why drift, not bugs, is the thing that kills your build. Template included.

This is Chapter 1 of ten, free. The full manual, with the companion prompt files and checklists, ships at micahjonesconsulting.com/playbook. Launch price \$99.

© 2026 MICAH JONES • FREE CHAPTER • SHARE IT, DON'T SELL IT